

Screenshot of Test Results

integerToRoman

- ✓ should convert 1 to "I"
- ✓ should convert 3999 to "MMMCMXCIX"
- ✓ should convert 3333 to "MMMCCCXXXIII"
- ✓ should convert 1444 to "MCDXLIV"
- ✓ should convert 3999 to "MCDXLIV"
- ✓ should convert 6 to "VI"

```
expect(integerToRoman(6)).toEqual("VI");
```

- ✓ should convert 77 to "LXXVII"
- ✓ should convert 828 to "DCCCXXVIII"
- ✓ should convert "2 to "II"
- ✓ should throw an error for numbers less than 1
- ✓ should throw an error for numbers more than 4000
- ✓ should throw an error for numbers less than 0
- ✗ should throw an error for invalid integer number because ?

```
AssertionError: expected [Function] to throw an error  
at n.<anonymous> (tests.js:56:47)
```

- ✗ should throw an error for invalid integer number because I

```
AssertionError: expected [Function] to throw an error  
at n.<anonymous> (tests.js:60:47)
```

- ✓ should throw an error for invalid less than 1 integer number because Empty
- ✓ should throw an error for numbers less than 1 because Space

romanToInteger

- ✓ should convert "I" to 1
- ✓ should convert "MMMCMXCIX" to 3999
- ✓ should convert "MMCMXCIX" to 2999
- ✓ should convert "VI" to 6
- ✓ should convert "LXXVII" to 77
- ✓ should convert "DCCCXXVIII" to 828
- ✓ should convert "IV" to 4
- ✗ should convert "II" to 2

```
Error: The Roman numeral contains invalid characters.  
at romanToInteger (script.js:60:11)  
at n.<anonymous> (tests.js:102:12)
```

- ✓ should convert "i" to 1
- ✗ should convert "i" to 1

```
Error: The Roman numeral contains invalid characters.  
at romanToInteger (script.js:60:11)  
at n.<anonymous> (tests.js:110:12)
```

- ✗ should convert "j" to 1

```
Error: The Roman numeral contains invalid characters.  
at romanToInteger (script.js:60:11)  
at n.<anonymous> (tests.js:114:12)
```

- ✓ should throw an error for empty input
- ✓ should throw an error for space input
- ✓ should throw an error for integer input
- ✓ should throw an error for integer+char input
- ✓ should throw an error for an input over 4 Equal parts
- ✓ should throw an error for an input over 2 Equal 5 parts
- ✓ should throw an error for an input where 10 is over 1000
- ✓ should throw an error for Non-Existent input

Failing Tests

The First Two Errors refer to the fact that while those results truly give an Error when Trying it in the Program the Errors Occur outside of the function making it impossible for the automated Tester to detect them which could be solved by activating the error inside the function.

The Third error is as well related to the fact that while the Spaces are not included in the verification of the String the program can do the calculations anyway leading to the error not Appearing in the Computer String of course this could be solved by including ' ' as a Valid Character.

Lastly the Two Ones are simply treated as wrong because there doesn't exist a remove "tilde" function unlike the turn to Uppercase function used so while i is turned into the valid Character I, Í has no transformation of course the solution is to make that transformation.